# Parametric CAD as an Action Space for Language-Model Agents: Design Tools That Can Check Their Own Work

**Gwilherm Kerherve**[1]
Department of Materials,
Imperial College London,
Royal School of Mines, South Kensington,
London SW7 2BP, United Kingdom
email: g.kerherve@ic.ac.uk

*Language models can now write CAD scripts from a description, but they write blind: a script that compiles may still put a hole on the wrong face, leave two parts overlapping or produce a wall too thin to print, and the model has no way of finding out. We argue that the useful unit is not the generated script but the loop in which an agent builds, inspects and corrects. We present KherveCAD, an open-source modeller in which every design is a tree of objects that maps one-to-one onto an OpenSCAD program, so that edits made by a person with the mouse and edits made by an agent as code act on the same document. The application exposes its operations to agents through the Model Context Protocol: tree edits, code that is parsed back into editable objects, rendered views the agent can look at, and measurements — mass properties, printability, interference and surface probes — it can compare against the brief. We describe the representation, the tool interface and the safety model, and set out an evaluation on a benchmark of design tasks that compares an agent writing code once, an agent that can see renders, and an agent that can also measure. [Summarise the measured outcome here in one sentence.]*

*Keywords: computer-aided design, large language models, AI agents, OpenSCAD, Model Context Protocol, design verification, parametric modelling*

## 1 Introduction

Engineers and scientists need custom parts all the time: brackets, adapters, fixtures, sample holders. Script-based CAD such as OpenSCAD [1] and CadQuery [2] suits this work, because a part becomes a short program that can be versioned and changed by editing one number. The same property makes these languages a natural target for large language models, and a growing body of work generates CAD from text, either as sketch-and-extrude sequences [3,4] or as scripts [5,6].

Most of this work treats generation as a single shot: a prompt goes in, code comes out, and the code is scored once it has been compiled. In practice the failures that matter are geometric, not syntactic. A script may run and still describe the wrong object. Alrashedy et al. [7] showed that letting a vision-language model look at renders of its output and revise the code improves results, and Sadik and Bujny [8] argued that geometric measures are needed to judge generated models at all. Both point to the same gap. A model that can only see pixels can tell that a part looks wrong, but not that a wall is 0.6 mm thick or that two parts intersect by a fraction of a millimetre.

This paper asks what a CAD system has to offer an agent so that the agent can check its own work. Our answer has three parts:

(1) **A shared, editable representation.** The design is a tree whose nodes map one-to-one onto OpenSCAD statements. An agent's code is parsed into the same tree a person edits with the mouse, so nothing the agent produces is a dead end.
(2) **Actions at the level of design intent.** Beyond writing code, the agent can insert library parts, mate Objects, fillet a chosen edge or build a crystal surface, each as one undoable step.
(3) **Observations that carry numbers.** Besides rendered images, the agent can ask for volume and centre of mass, a printability report, an interference test or the point where a ray meets a surface.

We implement this in KherveCAD, an open-source application [9], and describe an evaluation that separates the contribution of seeing from that of measuring.

## 2 Related Work

*Text-to-CAD..* CAD-LLM [3] fine-tuned a language model on engineering sketches. Text2CAD [4] generated parametric sketch-and-extrude sequences from prompts graded from beginner to expert. Text-to-CadQuery [5] and CAD-Coder [6] generate CadQuery scripts, the latter with chain-of-thought and a geometric reward. All evaluate a generated artefact against a target.

*Feedback and verification..* CADCodeVerify [7] closes the loop with a vision-language model that asks and answers questions about renders. Sadik and Bujny [8] propose volumetric, surface and dimensional metrics for generated models with a human in the loop. Our work differs in where the feedback comes from: the checks are tools of the CAD system, available to the agent on demand, rather than a fixed critic outside it.

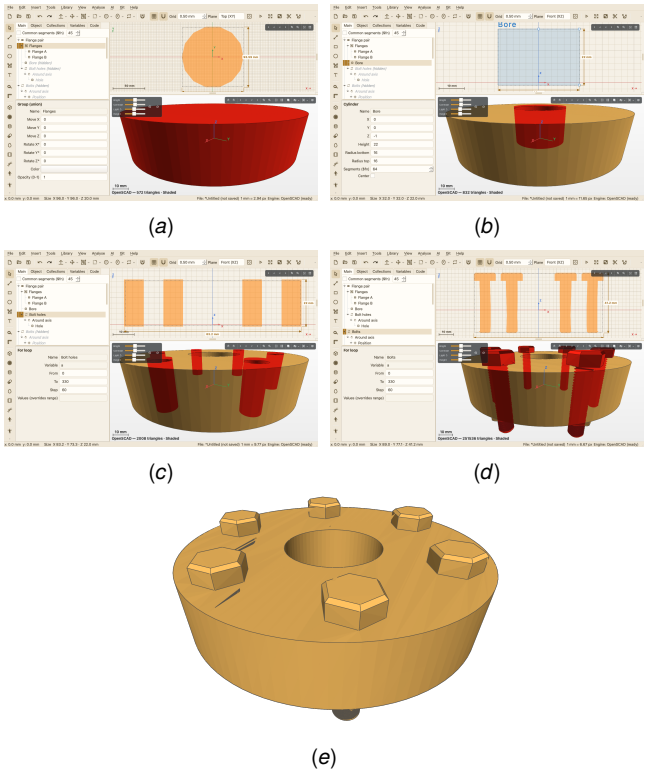
*Agent tool protocols..* The Model Context Protocol [10] is an open standard by which an assistant discovers and calls tools exposed by an application. It has made it practical to connect general assistants to desktop software, but we are not aware of an evaluation of what a CAD application should expose through it.

## 3 Representation

**3.1 One Tree, One Program.** Every KherveCAD document is a tree. Two-dimensional shapes, primitives, extrusions, transforms, Boolean operations, loops, conditionals and variables are nodes, and each node emits exactly one OpenSCAD statement.

[1] Corresponding Author.
October 12, 2026

 Copyright © 2026 by ASME 



(*a*) (*b*) (*c*) (*d*) (*e*)

**Fig. 1 A bolted flange joint built node by node: (a) two discs, (b) the bore added to a difference, (c) six bolt holes from one loop, (d) bolts and nuts, (e) the exact OpenSCAD render. The selected node is tinted red.**

The program is regenerated from the tree after every change. A program typed or sent by an agent is parsed by a recursive-descent reader into the same nodes, including user modules, functions and list comprehensions. A round trip from tree to code to tree is lossless and is part of the test suite. Comments name nodes, so a part labelled `// Bore` in code appears as "Bore" in the tree.

Figure 1 shows a bolted flange joint being built this way: each step adds one node, and the 3D view updates at once.

### 3.2 Parts and Assemblies.

An *Object* is a part definition that compiles to an OpenSCAD `module`; the main tab holds placed instances of Objects. Instances can be mated face to face, concentrically, as a hinge or through a gear ratio, and the mates are solved into ordinary placement parameters, so the program stays plain OpenSCAD. Figure 2 shows a printable butt hinge as three Objects, and Listing 1 the program they produce.

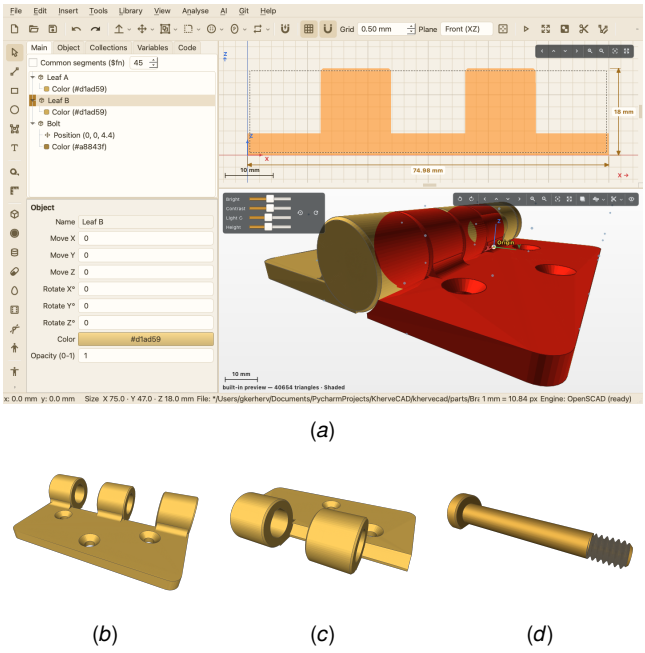
### 3.3 Geometry.

A Python tessellator draws an immediate preview, with Boolean operations cut by Manifold [11] when it is installed. OpenSCAD then renders each Object separately in the background; the exact mesh replaces the preview as it arrives and is cached by the Object's content. Rendering per part keeps each part's colour and means that moving a part re-renders nothing. Exchange with other CAD systems goes through Open CASCADE [12], which writes exact STEP surfaces for primitives, extrusions, revolutions and Booleans.

## 4 The Agent Interface

KherveCAD exposes 88 tools through MCP (Fig. 3). Table 1 groups them by what they give the agent.

Three design rules shaped the interface. First, every action is an action a person could take, through the same code path; the agent never edits a private copy. Second, each tool call is one undo step, so a person can step back through an agent's work with Ctrl+Z. Third, the agent is told why something failed: parameter names are checked against the node schema, a program that yields no objects is refused rather than silently emptied, and an export that could only be approximated says so. The server listens only on the local machine with a per-session token, and the user chooses whether the agent may read, edit the open document, or also touch files.



(*a*) (*b*) (*c*) (*d*)

**Fig. 2 Butt hinge in three printable Objects: (a) the tree, (b) Leaf A, (c) Leaf B, (d) the removable threaded pin.**

**Listing 1:** Outline of the program generated for the hinge. Each Object is a module and one placed call.

```
module Leaf_A() { ... } // 97 lines
color("#d1ad59") Leaf_A(); // Leaf A
module Leaf_B() { ... } // 74 lines
color("#d1ad59") Leaf_B(); // Leaf B
module Bolt() {
    difference() {
        union() {
            translate([0, 0, 4.6]) rotate([0, 90, 0]) // head
                rotate_extrude($fn=96) polygon(points=[...]);
            translate([-0.5, 0, 4.6]) rotate([0, 90, 0]) // shank
                cylinder(h=59, r1=4.95, r2=4.95, $fn=96);
            translate([58, 0, 4.6]) rotate([0, 90, 0]) // thread
                linear_extrude(height=14.5, twist=-2610, slices=174)
                    polygon(points=[...]);
        }
        translate([-7, -10, -10]) cube([80.5, 20, 10]); // flat to print on
        translate([-6.01, 0, 4.6]) rotate([0, 90, 0]) // hex socket
            cylinder(h=5.01, r1=3.4641, r2=3.4641, $fn=6);
    }
}
color("#a8843f") translate([0, 0, 4.4]) Bolt();
```

## 5 Evaluation

### 5.1 Research Questions.

RQ1 Does an agent that can inspect renders produce correct parts more often than one that writes code once?

RQ2 Do numerical observations (mass properties, printability, interference) add to what renders alone provide?

RQ3 Does building into an editable tree, rather than writing a script, change how often the result can be modified afterwards without starting again?

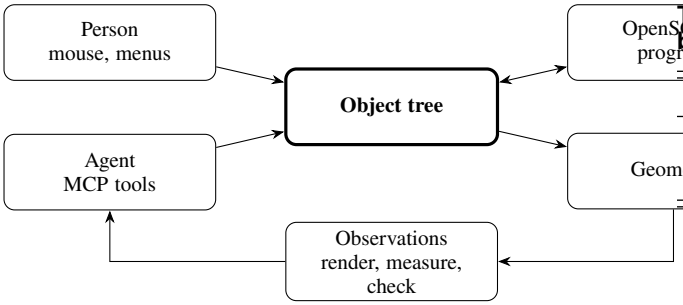



**Fig. 3 The person and the agent edit the same tree; the agent also receives observations of the geometry.**

**Table 1 Tool groups offered to the agent (selection).**

| Group | Examples |
|---|---|
| Program | `apply_code` (OpenSCAD parsed into nodes; refused if nothing parses), `get_code`, `check_code` |
| Tree edits | `add_node`, `set_params`, `wrap_nodes`, `make_object`, `attach_parts` |
| Design-intent actions | `insert_part` (library), `fillet_edges`, `push_pull_face`, `split_part`, `build_surface` |
| Seeing | `render_view` (image from any camera, after the exact render) |
| Measuring | `mass_properties`, `check_printability`, `check_interference`, `probe_surface`, `measure`, `section` |
| Documents | `save_document`, `export_document`, `export_drawing` |

**5.2 Tasks.** The benchmark holds [N] tasks in three tiers: single parts with stated dimensions (e.g. an L-bracket with two counterbored M5 holes), printable parts with constraints (a hinge with 0.3 mm clearance and no overhang above 45°), and assemblies (a flange pair bolted on a pitch circle with no interference). Each task has a written brief, a reference model and an automatic checker.

**5.3 Conditions.** The same model is run in three conditions: **C0**, code written once and submitted; **C1**, the agent may call `render_view` and revise; **C2**, the agent may also call the measuring tools. Each task is repeated [k] times per condition, with a budget of [B] tool calls.

**5.4 Metrics.** Success means the result passes every check in the brief. We also report the volume intersection-over-union with the reference, the largest dimensional error, the number of tool calls, wall-clock time, and, for RQ3, whether a follow-up change ("make the plate 5 mm thicker") succeeds in one call.

**5.5 Results.** [Report Table 2 and the failure analysis here.]

## 6 Applications

KherveCAD is used to teach CAD modelling: 21 graded tutorials take a student from a single cube to loops and modules, and the code tab shows the program growing with each click. PhD students use its crystal and surface builders to prepare unit cells and slabs; a bulk-terminated surface along any $(hkl)$ is one call and stays an editable program (Fig. 4a). The same representation holds a full-size ultra-high-vacuum chamber, which we used to check how a new system would fit in our laboratory, and architectural models such as a Regency town house built with the House Builder (Fig. 4).

KherveCAD is part of KherveTools [13], a family of free, open-source applications for scientists that includes KherveFitting [14].

**Table 2 Results by condition. [To be filled from the benchmark runs.]**

| Condition | Success (%) | IoU | Max error (mm) | Calls |
|---|---|---|---|---|
| C0 code once | – | – | – | 1 |
| C1 + renders | – | – | – | – |
| C2 + measurements | – | – | – | – |



(*a*) (*b*) (*c*)

**Fig. 4 One representation across scales: (a) a carbon nanotube, (b) a UHV chamber mock-up, (c) a Regency house.**

## 7 Discussion and Limitations

The interface makes errors visible; it does not prevent them, and results depend on the model behind the agent. The built-in preview approximates some operations until OpenSCAD has rendered them, so an agent checking too early may measure an approximation; `render_view` waits for the exact render by default for this reason. Sketches have no geometric constraint solver yet, so design intent such as "tangent" or "equal" has to be written as expressions. The benchmark covers mechanical parts and does not test freeform surfaces.

## 8 Conclusions

Giving an agent a CAD system rather than a code box means giving it the same actions a person has, on the same document, together with the observations a person would use to check a part. KherveCAD implements this through a tree that maps one-to-one onto OpenSCAD and an MCP interface of 88 tools. [Restate the measured gain from C0 to C2.] The software and the benchmark are open source.

## Software Availability

KherveCAD is released under GPL-3.0 at https://github.com/gkerherve/KherveCAD and archived on Zenodo [9].

## Acknowledgment

[To fill in.]

## Declaration of Generative AI Use

The author used Claude (Anthropic) to help write code and draft text, and reviewed and edited all content.

## References

[1] Kintel, M. and Wolf, C., "OpenSCAD: The Programmers' Solid 3D CAD Modeller," https://openscad.org, Accessed 12 October 2026.

[2] CadQuery contributors, "CadQuery: a Python parametric CAD scripting framework based on OCCT," https://github.com/CadQuery/cadquery, Accessed 12 October 2026.

[3] Wu, S., Khasahmadi, A., Katz, M., et al., 2023, "CAD-LLM: Large Language Model for CAD Generation," *NeurIPS 2023 Workshop on Machine Learning for Creativity and Design*.

[4] Khan, M. S., Sinha, S., Sheikh, T. U., Stricker, D., Ali, S. A., and Afzal, M. Z., 2024, "Text2CAD: Generating Sequential CAD Designs from Beginner-to-Expert Level Text Prompts," *Advances in Neural Information Processing Systems*, Vol. 37, doi: 10.52202/079017-0242.

[5] Xie, H. and Ju, F., 2025, "Text-to-CadQuery: A New Paradigm for CAD Generation with Scalable Large Model Capabilities," 2505.06507

[6] Guan, Y., Wang, X., Xing, X., Zhang, J., Xu, D., and Yu, Q., 2025, "CAD-Coder: Text-to-CAD Generation with Chain-of-Thought and Geometric Reward," *Advances in Neural Information Processing Systems*, arXiv:2505.19713.

[7] Alrashedy, K., Tambwekar, P., Zaidi, Z., Langwasser, M., Xu, W., and Gombolay, M., 2025, "Generating CAD Code with Vision-Language Models for 3D Designs," *International Conference on Learning Representations (ICLR)*, arXiv:2410.05340.

[8] Sadik, A. R. and Bujny, M., 2025, "Human-in-the-Loop: Quantitative Evaluation of 3D Models Generation by Large Language Models," 2509.07010

[9] Kerherve, G., 2026, "KherveCAD: a parametric CAD application with OpenSCAD as its engine, driven visually or by Claude (v0.1.535)," doi: 10.5281/zenodo.23305286.

[10] Anthropic, 2024, "Model Context Protocol Specification," https://modelcontextprotocol.io, Accessed 12 October 2026.

[11] Lalish, E. et al., "Manifold: a geometry library for topological robustness," https://github.com/elalish/manifold, Accessed 12 October 2026.

[12] Open Cascade SAS, "Open CASCADE Technology," https://dev.opencascade.org, Accessed 12 October 2026.

[13] Kerherve, G., "KherveTools: applications made by a scientist, for scientists," https://khervetools.com, Accessed 12 October 2026.

[14] Kerherve, G., Skinner, W. S. J., Hochhaus, J. A., Graf, A., Morgan, D. J., Isaacs, M. A., Reed, B. P., Nakajima, H., and Payne, D. J., 2026, "KherveFitting: An open source software for fitting X-ray photoelectron spectroscopy data," Surface and Interface Analysis, **58**(1), pp. 54–65.

## List of Figures


1 A bolted flange joint built node by node: (a) two discs, (b) the bore added to a difference, (c) six bolt holes from one loop, (d) bolts and nuts, (e) the exact OpenSCAD render. The selected node is tinted red. . . . . 2
(*a*) . . . . . 2
(*b*) . . . . . 2
(*c*) . . . . . 2
(*d*) . . . . . 2
(*e*) . . . . . 2
2 Butt hinge in three printable Objects: (a) the tree, (b) Leaf A, (c) Leaf B, (d) the removable threaded pin. . . . . 2
(*a*) . . . . . 2
(*b*) . . . . . 2
(*c*) . . . . . 2
(*d*) . . . . . 2
3 The person and the agent edit the same tree; the agent also receives observations of the geometry. . . . . 3
4 One representation across scales: (a) a carbon nanotube, (b) a UHV chamber mock-up, (c) a Regency house. . . . . 3
(*a*) . . . . . 3
(*b*) . . . . . 3
(*c*) . . . . . 3


## List of Tables


1 Tool groups offered to the agent (selection). . . . . 3
2 Results by condition. [To be filled from the benchmark runs.] . . . . . 3